A Case Study in Performance, Modifiability and Extensibility Analysis of a Telecommunication System Software Architecture *

Marco Castaldi, Paola Inverardi
Dipartimento di Informatica
Università dell'Aquila
via Vetoio 1, 67010 Coppito, L'Aquila, Italy
[castaldi, inverard]@di.univaq.it

Sharareh Afsharian
Dipartimento TSA
Ericsson Lab Italy SpA
via Anagnina 203, 00040, Roma, Italy
shary.afsharian@eri.ericsson.se

Abstract

In this work we present a case study in architecture analysis. The analysis was conducted on a real system, the Ericsson product "GSM On the Net", in order to evaluate two alternative software architectures with respect to a set of attributes required by the customer namely extensibility, performance and modifiability. The two architectures differ because of the use of different internal protocols, SIP versus H323, that deeply influence the structure of the system architecture.

1. Introduction

In this work we describe the result of a project between ERI (Ericsson Lab Italy) and University of L'Aquila to address some problems of GSM on the Net (GON), an Ericsson system project already designed and developed, which at the time of the analysis was at release 2.0.

This system presented different kinds of problems: the first one was (1) a performance problem, due to the telecommunication protocol used internally, which requires many messages to be exchanged in order to work; then the system appeared to be (2) non extensible and (3) non modifiable. As a consequence of the last two problems it was difficult to add new supplementary services in a easy and quick way. This could in turn result in a violation of strict *time-to-market* requirements.

The performance problem was in part solved by the GON team using some implementation tricks inside a component, without modifying the general architecture of the existing system, whereas the extensibility and modifiability problems were identified as architectural problems, meaning that in order to fix them, the software architecture of the system had to be changed.

The solution, according with the GON team, was to replace ITU-T H.323 [1], the currently used telecommunication protocol, with SIP by IETF [9]. SIP is a new protocol for telecommunications over IP that allows an easy and effective implementation of supplementary services [17].

We designed a new software architecture for GON based on SIP, with the aim to increase modifiability and extensibility of the system, and overall the system performances. Then, we evaluated the two different architectures, proving that the new one is more modifiable, that allows for easy extension with respect to supplementary services and that guarantees better performances.

In the context of complex and large telecommunication systems, the results of this work show how analysis of quality attributes such as performance, modifiability and extensibility directly at software architecture level can help designers and managers to take crucial choices for the system, like the adoption of a particular protocol.

Although recently there have been several approaches [5, 3, 15, 12, 13, 21] that addressed the problem of performance analysis at architectural level and there is global consensus on the importance of performance analysis at early stages of software development in order to support important design decisions, to perform architecture evaluation with respect to quality attributes it is still difficult in a industrial context.

This depends on several factors among which there are two well-known key elements. The first one is due to a lack of knowledge in how to conduct this kind of studies. The second one is a lack of consolidated methods and tools that can help designers and architects to carry out quantitative analysis by taking advantage of the software artifacts that already exist in the system life-cycle.

In Section 4, we discuss these issues in more depth together with the reasons that drove our choices in the context of this particular case study.

*The authors would like to acknowledge the ERI manager Emilia Peciola and the technical coordinator Giovanni Nucera, and the Italian M.U.R.S.T. national project SALADIN that partly supported this work.

Proceedings of the 10th IEEE Int'l Symp. on Modeling, Analysis, & Simulation of Computer & Telecommunications Systems (MASCOTS'02)
1526-7539/02 $17.00 © 2002 IEEE
Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

In this paper, for space reasons, we present a short description of the complete work. The complete version is in [7].

In Section 2 we describe the Ericsson system, the H.323 protocol architecture and the system architecture based on H.323. In Section 3, after a brief description of the SIP protocol architecture, we show the new software architecture based on SIP.

Section 4 is dedicated to the software architecture evaluation scenarios-based methodology that we have used, while in Section 5 we show, through UML sequence diagrams, the behavior of the two architectures in one of the scenarios selected for the evaluation. In Section 6 we analyze the results of the evaluation with respect to performance, modifiability and extensibility.

2. The Ericsson System

GSM on the Net (GON) wants to provide a global solution for the communication in the enterprise environment [20].

There are different technologies to manage sectorial communications in an enterprise: PBX and PSTN phones for internal and external voice communications, DECT and GSM phones for internal and external mobile communications, Internet applications for video conferencing, for email exchanging and so on. The GON system offers a unified platform able to provide the most popular data networking technologies and the most popular digital wireless standards [20].

The kernel of the system is the enterprise LAN: GON offers voice, data and multimedia services over the company's Intranet.

The GON system is based on the ITU-T H.323 recommendation [1], that is following described.

2.1. The H.323 protocol architecture

The H.323 protocol is an umbrella of ITU recommendations that specifies all the different protocols that are used to manage terminals, equipments and services for multimedia communications over a packet based network. In particular, it specifies Network Control with H.225, Session Control with H.245, Media Coding with H.26x and G.7xx, Gateways with H.246, Security with H.235 and Supplementary Services with H.450 [1].

The H.323 protocol architecture defines: (i) the **Terminal,** which is an audio or video equipment, or a software program, that understands the H.323 logic. All terminals must support voice communication, while video and data are optional, (ii) the **Gateway:** is the connection point between H.323 terminals and PSTN terminals, GSM terminals and others. In the H.323 gateway the translation between different signaling protocols occurs. (iii) The **Gatekeeper,** performs call control functions, such as Address Translation, Admission Control, Management of Gateway, Call Signaling, Call Management, Bandwidth Control, Charging and Billing. This is the kernel of a H.323 network. The (iv) **Multipoint Control Unit,** controls and manages a multipoint conference, i.e. a call with three or more terminals.

H.323 is a very good protocol, in fact it is the most used protocol to create multimedia systems over IP. But this protocol presents different problems: ***a binary codification*** which reduces the error detection and recovery, an ***hard call control signaling***, this means that all control messages have to pass through the Gatekeeper, increasing the complexity of the communication. Other problems are an ***expensive call setup***, given that to set up a communication needs the exchange of a lot of messages, a ***strict use of transport layer***, only TCP is used, and ***low integration*** with other Internet protocols, like HTTP or SMTP.

2.2 Software architecture of GSM on the Net

GSM on the Net is a system devised for Internet Service Providers (ISP) and GSM operators, which will increase the offering of services for the enterprise communications.

The components of the system architecture reside either on the ISP LAN and on the Enterprise LAN. This situation forces the introduction of Firewalls, for security control, specialized Proxies, that provide a masquerading of IP packets for H.323, RTP and GSM messages, and a duplication of Application Node (a set of API to allow the interface with external applications) and Voice Gateway (a component that translates the PSTN or PBX protocol to H.323 protocol) over both LANs [20]. The Figure 1 illustrates this architecture.

In the End Customer LAN there are:

- **BTS:** this is a usual Base Station for GSM radio transmission.

- **A-Bis Gateway (AGW):** this is a component that packs the GSM messages into IP packet, to allow the transport over the IP network. It is important to notice that in the AGW there is not a protocol translation from GSM protocol to H.323 protocol, but GSM messages pass directly over the IP net.

- **Terminal Agent (TA):** a Terminal Agent is the component that allows all terminals H.323 to access the system. The most important functionality of TA is the multiplexing of message requests and responses from/to the system's kernel, on the ISP LAN. Using H.323 the media packets must be sent over TCP: the management of a TCP connection is very expensive,

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

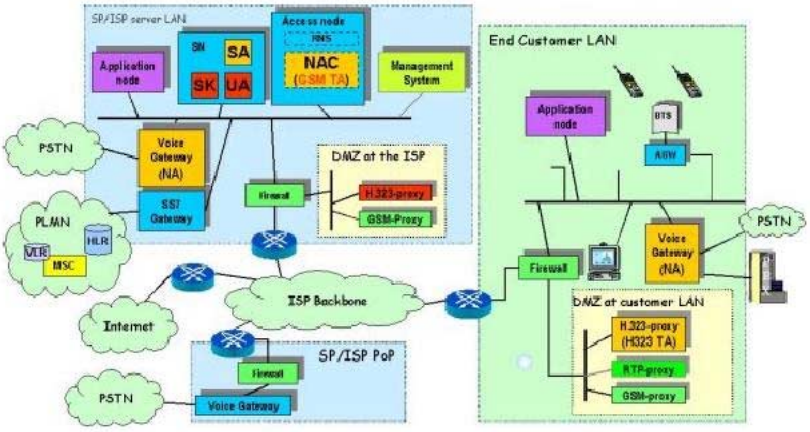

Figure 1. Software Architecture of GSM on the Net in the ISP scenario

therefore in the TA there are a great number of connections always active that send the messages when necessary [20].

In the ISP LAN there are:

- **Access Node:** the control node for the GSM terminals. It is composed by:

  Radio Network Server (RNS): it provides radio network management and traffic functions for the GSM terminals.

  Network Access Controller (NAC): this is a Terminal Agent for the GSM terminals. Here the system provides the protocol translation between GSM proprietary protocols and H.323 protocols, i.e. a GSM terminal becomes a H.323 terminal.

- **Service Node (SN):** this is the main component of the system. The SN is composed by:

  User Agent (UA): implements the users system functionalities, such as User Authentication, User Registration, Administration of user identifiers, User and Service Mobility, H.323 Call handling, Service Triggering, Interface with the DB containing user information and profiles, Interface with charging and billing services [20].

  Service Agent (SA): provides the implementations of supplementary services, such as Virtual Private Network, Calling Line Identity Presentation/Restriction, Call Forwarding, Call Barring (H.245.x).

  SiteKeeper (SK): the SK is the interface between the SN and the system access provider. All terminals (if necessary converted to H.323 by an access agent) enter in the SN through the SK. The SK performs the routing for the calls and resources management. With resources management we intend the capabilities exchange in the H.323 call and the global bandwidth management in the system [20].

- **Management System:** management features for the system.

- **SS7 Gateway:** it is the gateway dedicated to PLMN interface.

It appears clear that the software architecture of GON is an instance of H.323 protocol architecture.

3 The new software architecture of GSM on the Net

To address the mentioned problems, it has been decided to replace the ITU-T H.323 internal protocol with IETF SIP (Session Initiation Protocol) [9, 16].

With SIP protocol, many specific problems of the H.323 are automatically solved, and the system acquires all the interesting properties of this new protocol, text based and HTML-like [16].

Given the new protocol components, trying to reuse as much as possible of existing components, and paying attention to the software architecture problems presented above, we have designed a software architecture trying to satisfy the requirements modifiability and extensibility, to improve the performances and the easiness of inserting new supplementary services [18].

In this section we first introduce the SIP architecture and then we present the new GON architecture.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

3.1. The SIP architecture

The Session Initiation Protocol (SIP) by IETF is a protocol specified in the 1999, with the RFC 2543. It is proposed like a standard for multimedia communication over IP network.

SIP is a text based protocol, specified using the HTTP ideas. Referring to the H.323 protocol problems described in section 2.1, SIP presents some greatest advantages: it is ***a text based protocol***, given that the messages are similar to HTTP messages, with Requests and Responses, composed by headers. With a textual specification, there are optimal error detection and recovery, and the parser implementation is very easy. The protocol is ***independent from the transport layer*** (SIP is over TCP and UDP), it has ***easy routing*** with a naming email like (for example, `sip:castaldi@di.univaq.it` to indicate the user `castaldi` on the sip domain `di.univaq.it`) that uses dns for network routing. Finally, SIP provides a ***very good integration*** with other IETF protocols, as SMTP e HTTP, and it is particularly suitable to create an intelligent network, with supplementary services [17].

SIP specifies the following architectural components:

- **User Agent:** it is the SIP component that acts as user in a SIP system. It is composed by an User Agent Client (UAC) that sends the requests or responses, and an User Agent Server (UAS) that receives requests or responses from other components. In this work we refer to User Agent SIP with the couple UAC/UAS. The UAC/UAS maintains the state of call.

- **Proxy Server:** this component performs the location of called terminal (callee), this means the association between SIP URL and IP address where run the called UAC/UAS. There are two possible kinds of proxy:

  Stateful Proxy: it maintains the call state, it implements the UAC/UAS functionality to send and receive message requests and responses, and it provides the requests forking for parallel research of the callee.

  Stateless Proxy: performs terminal location, message sorting and it does not know the call state.

- **Redirect Server:** this server realizes the user mobility. If the user is placed at the moment in an other site, the request received by the Redirect Server is sent to the location specified by the user.

- **Registrar Server:** this is the server dedicated to terminal registration.

3.2. The new architecture for GoN

With the SIP components, we have to build the new software architecture for the GON system, by maintaining the same functionalities of the previous version. Figures 2 illustrates the new architecture in the ISP scenario.

The **Terminals** (IP phones, PC clients, GSM phones), **Management Node**, **Application Node**, **BTS**, **A-Bis Gateway** and **Firewall** have not been changed, while the **SIP and RTP Masks** have the same functionalities of the Proxies in the old architecture, but a different name to distinguish them from the Proxy Server specified by SIP. Note that there is not a GSM mask, given that the GSM messages don't go out of End Customer LAN, but they are converted into SIP messages by the Multiprotocol Access Controller (see later).

The components that we have changed or created in the End Customer LAN are the following:

- **Voice Gateway:** this component is different from the same named component in the old architecture. The Voice Gateway has the same function of A-Bis Gateway, i.e. to pack the PSTN, PLMN and PBX specific protocols signaling messages into IP packets in order to be carried over a LAN. There is no protocol translation.

- **Multiprotocol Access Controller (MAC):** MAC is the access point to the system for the different terminals. Every terminal that wants to enter the system has to connect to the MAC, which provides the protocol translation and it supplies SIP logic to terminals. MAC is composed of three different levels:

  Gateway level: the access point for the terminals to the MAC. At this level there are all the gateways that perform the protocol translation. In our case there are H.323 GW (at this moment the internal system protocol is SIP), PSTN GW, PLMN GW, and GSM GW.

  HLTP level: the previous gateways perform a translation between the proprietary protocols to High Level Translation Protocol (HLTP). This protocol understands the type of operation that the terminal wants to execute, and it informs the higher level. For example, when a H.323 terminal wants to register, it sends a RRQ message to the Gatekeeper, while a SIP terminal sends a REGISTER Request to the Registrar Server. Anyway both want to register, therefore the message captured by the Gateway level is translated to the HLTP REGISTRATION message.

  UAC/UAS level: given that we have decided to

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

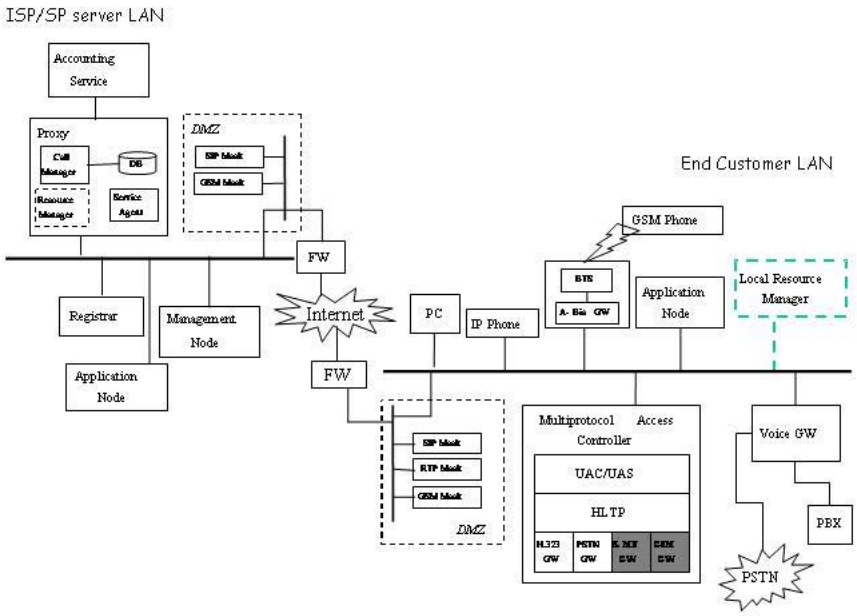

Figure 2. Proposed SIP-based architecture of GoN

use the SIP protocol in the system, over the HLTP level there is the UAC/UAS level, where the system associates a SIP User Agent (UAC/UAS) to each terminal. An HLTP message is translated to the respective SIP message, handled by a UAC/UAS.

- **Local Resource Manager**: manages the resources of local network. The SIP protocol does not take care of resources management (bandwidth), so this functionality has to be handled explicitly.

On the ISP LAN there are:

- **Registrar Server**: this is the server that manages the registration of the SIP terminals.
- **Accounting Service**: it is the service that manages charging and billing.
- **Proxy Server**: this is a stateful Proxy Server. It is composed of:

  Call Manager: this component manages the SIP call. The user administration is performed by the Call Manager, using the internal Database. The Call Manager makes the forking of the requests, and it maintains the information for charging and billing. This component performs the terminal location.

  Resource Manager: if the local resource managers on the different End Customer LANs are implemented, the resource manager is the component that coordinates and summarizes all the information.

  Service Agent: this component allows the supplementary services implementation [17].

  Database: it contains all the information about the users, to make authentication, profiling etc.

4. The architecture evaluation

In this paper we will evaluate quality attributes significantly different each other. We use the definition reported in [6] where the Non-Functional requirements (NFRs) are categorized in *development NFRs* and *operational NFRs*. In particular modifiability and extensibility are classified in the first category and performance in the second one. For this reason, we used different metrics to perform the evaluation.

There are many interesting works which describe software architectures analysis for several quality attributes. The most popular and used approaches to evaluate software architectures of industrial products are scenario-based, such as SAAM [12] and ATAM [13]. We decided to use for the GON architecture evaluation a scenario-based approach, but more informal than [12] or [13]. In particular, we used the

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

five steps evaluation proposed by Bosch and others in [5]:

Step 1: *Set goal*. The goals for the evaluation are performance and extensibility/modifiability with respect to supplementary services. This kind of modification is categorized by Oskarsson in [14] as *Extension of capabilities*, and it makes the methodology described in [5] not appropriate, since it is too much general.

Step 2: *Describe software architecture*. The architecture is described using UML diagrams, in particular Sequence, State and Activity Diagrams, given that the existing architecture for GON system based on H.323 protocol is completely described using this notation. We also use some informal Collaboration Diagram to show component relationships and communication. As consequence of this choice the approach proposed in [12], based on a small Architecture Description Language (ADL), is not directly applicable.

Step 3: *Elicit scenarios*. The choice of the scenarios to evaluate was driven by the system domain. GON is a telecommunication system, it means that the most important and significant part of its work is to make calls. Therefore we chose as evaluation scenarios the "call with services" and the "basic call", that is a call between two terminals with and without supplementary services, respectively.

Step 4: *Evaluate scenarions*. The evaluation of different scenarios is attribute dependent. For modifability, the most used evaluation metrics are based on the "identification of architectural elements affected by a change" [5], and on the "identification of the changes in the relationships between architecture components" [12, 13]. For performance analysis, the Ericsson GON Team chose the parameter "number of exchanged messages", given that some internal studies discovered that the main cause of performance problems for the system was the large number of exchanged messages. A more formal approach would be possible by using the Queuing Network Model approach proposed in [3], using the Message Sequence Charts instead of Labeled Transition Systems to model the system dynamics [2]. Unfortunately, the lack of a tool which performs this task makes the evaluation of this large Ericsson system not easily feasible. We decided to evaluate also the number of components involved in the communication, because this number is an important factor for the definition of the general performance of the system.

Step 5: *Interpret the results*. The detail of results is described in Section 6, using tables to compare the results for both architectures in each scenario.

For space reason, in this paper we show only the basic call scenario, whereas the scenario call with services is described in detail in [7].

5 A selected scenario: the basic call

It is common to split a call in different steps: the first step is the **Discovery**, where the Terminals involved in the communication (Caller and Callee) have "to discover" the component which manages the global communication (the GateKeeper in the H.323 protocol, in SIP this operation does not exist). Then, the terminals have a **Registration**, where they inform GK/Proxy that they are ready to begin or receive a call. The next step is the **Call Setup**, where the Caller finds the Callee, and the **Capabilities Exchange**, where the two terminals reach an agreement on the communication parameters (encoding type, IP addresses, etc). When Caller and Callee agree, the call starts. At the end of the conversation, each Terminal can **Release** the communication. When a terminal wants to detach from the network, it has to cancel his registration with the **Unregistration** procedure. An Activity/State Diagram that shows the general behavior of the basic call can be found in [7].

To understand the system behavior, we show the flow of control for a basic call in both architectures.

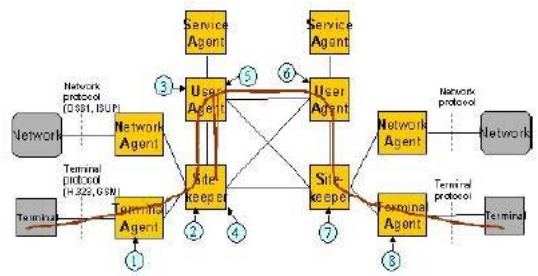

Figure 3. Flow of control for a basic call in the H.323 architecture

In the H.323 architecture, a Terminal connects to the Terminal Agent (1), which connects to the User Agent (3) through the SiteKeeper (2). The User Agent authenticates the Terminal, it finds (4, 5) the callee and it reaches the User Agent in the terminating side (6). The Callee User Agent reaches the Terminal through the SiteKeeper (7) and Terminal Agent (8). The Figure 3 represents this scenario.

The Figure 4 shows the SIP architecture, where a Terminal connects to the Multiprotocol Access Controller (MAC) (1), which sends the request to the Call Manager in the Proxy (2). The CM authenticates the Terminal and it checks the Registrar to find the callee (3). If the Callee is located over another domain, it sends (4) the request to the specified domain's Call Manager (5), which checks its Registrar to know the actual address. Finally, it reaches the terminal through the MAC (6).

Each call phase is described with a Sequence Diagram [4], that shows the messages exchanged between the com-

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

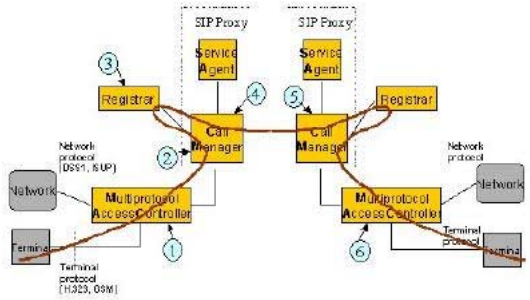

Figure 4. Flow of control for a Basic Call in the SIP architecture

ponents in both architectures. In the next sections we furnish the details of the communication, but for space reasons we cannot insert every sequence diagram. The missing diagrams can be found in [7].

5.1 Discovery

Each H.323 Terminal sends a RAS **GRQ** (Gatekeeper ReQuest) message to its Terminal Agent. This message, through the SiteKeeper, reaches the User Agent. The User Agent authenticates the Caller Terminal and responds with a RAS **GCF** (Gatekeeper ConFirmation) message, which reaches the Terminal through SiteKeeper and Terminal Agent.
H.323: *Component involved: 3, Exchanged messages: 2.*

Note that SIP does not schedule this phase.
SIP: *Component involved: 0, Exchanged messages: 0.*

5.2 Registration

Each H.323 Terminal sends a RAS **RRQ** (Registration ReQuest) message to its Terminal Agent. This message, through the SiteKeeper, reaches the User Agent. If the Terminal is not registered, the User Agent creates a registration and responds with a RAS **RCF** (Registration ConFirmation) message, that arrives to the Terminal through SiteKeeper and Terminal Agent.
H.323: *Component involved: 4, Exchanged messages: 2.*

In the SIP architecture, the Terminal sends a REGISTRATION (HLTP) request to the MAC. This request is translated in a **REGISTER** Request sent to the Registrar. The Registrar authenticates the Terminal; if it does not exist another registration from the same terminal it creates a new registration, and it responds with a **200 OK** message directly to the MAC. The MAC sends an aknowledgment to the Terminal.
SIP: *Component involved: 3, Exchanged messages: 2.*

5.3 Call Setup

The H.323 terminal that wants to start a call, sends a **Setup** message to the User Agent, through the associated Terminal Agent and SiteKeeper. Each component sends a **Call Proceeding** message when it receives a Setup message. When the Setup message reaches the User Agent, it checks the Terminal registration, and it searches the called terminal sending a RAS **LRQ** (Location ReQuest) message to the SiteKeeper. The SiteKeeper finds the callee and it responds to the UA with a RAS **LCF** (Location ConFirmation) message. So the UA sends the Setup request to the callee. The callee sends an **Alerting** message (ringing phone) and after a **Connect** message, which arrives to the Terminal through User Agent, SiteKeeper and Terminal Agent. Note that the User Agent sends a message to the accounting to start the billing. The sequence diagram SIP Call Setup is shown in Figure 5.
H.323: *Component involved: 8, Exchanged messages: 8.*

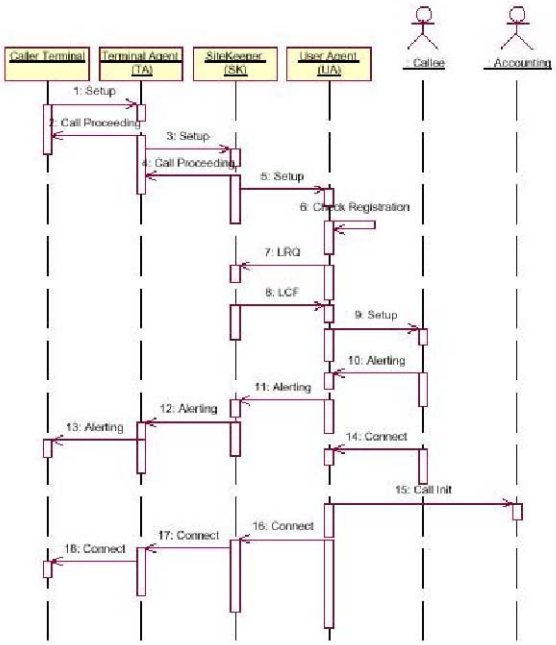

Figure 5. Sequence Diagrams for H.323 Call Setup in GoN

With SIP the Call Setup is very easy. The Terminal connects to the Gateway Level in the MAC. Here is provided protocol translation, and through a CALL START message by HLTP protocol, the UAC/UAS in the MAC sends an **INVITE** message to the Call Manager in the Proxy. This

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

message contains sip address of destination terminal and, especially, the SDP (Session Description Protocol) data, which allows the capabilities exchange. The Call Manager interacts with the Registrar to find the actual callee's address, and then it sends the modified (if necessary) INVITE message to the called Terminal. Each component sends a **100 Trying** message to the sender when it receives the INVITE message. While the called terminal rings, it sends a **180 Ringing** message to the Call Manager in the Proxy. This message is sent to the MAC where it is translated in a RINGING HLTP message and it reaches the Terminal. When the communication is established, the Callee sends a **200 OK** message to the CM in the Proxy, which activates the billing sending a message to the Accounting. Then, the 200 OK is sent to the MAC, it becomes a CALL ESTABLISHED HLTP message and it arrives to the Terminal. Finally, the caller MAC confirms the communication with a **ACK** message. The sequence diagram SIP Call Setup is shown in Figure 6.
SIP: *Component involved: 8, Exchanged messages: 8.*

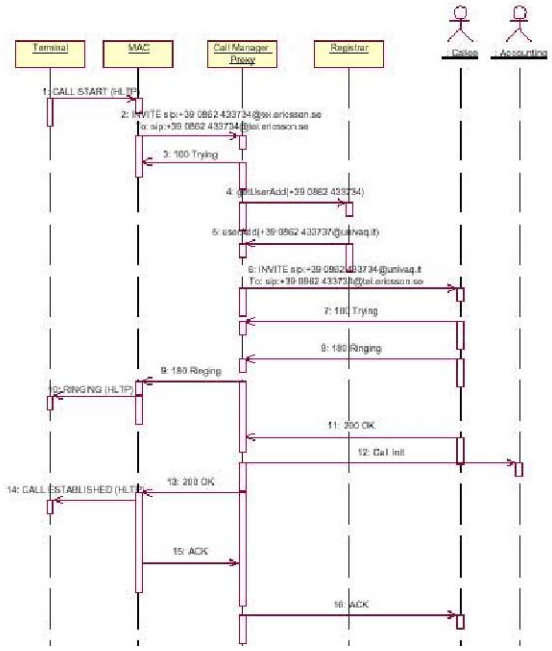

Figure 6. Sequence Diagrams for SIP Call Setup in GoN

5.4 Call Release

The call release is the last phase of a call: when the caller or the callee decide to finish the communication, they send a message to release the resources.

In H.323, the Terminal sends a **Release Complete** message to the Terminal Agent, the TA releases the local resources, and sends the Release Complete message to the SiteKeeper, which releases the global resources and sends the Release Complete message to the callee through the User Agent.
H.323: *Component involved: 8, Exchanged messages: 1.*

For SIP, the sequence is very similar. The Terminal sends RELEASE HLTP to the MAC. The MAC translates this message in a **BYE** message and sends it to the Proxy Server, which ends the billing for the caller and sends the BYE message to the Callee.
SIP: *Component involved: 6, Exchanged messages: 1.*

5.5 Unregistration

The Unregistration is the phase when the Terminal decides to detach from the network. In H.323, the Terminal sends a **URQ** (Unregistration ReQuest) to the Terminal Agent, which sends it to the UserAgent through the SiteKeeper. The SK checks if there is a call in progress, and if not, it removes the registration from the DataBase, After that, it sends back a **UCF** (Unregistration ConFirmation) to the Terminal Agent always through the SK, and the TA forwards the message to the Terminal.
H.323: *Component involved: 4, Exchanged messages: 2.*

For SIP, the Terminal sends a UNREGISTRATION HLTP message to the MAC, that translates it in a **REGISTER Expires: 0** message and sends it to the Proxy Server. The Proxy Server checks if the user is registred, if there is a call in progress, and then performs the unregistration. To confirm the unregistration, the Proxy Server sends back a **200 OK** message to the MAC, which reaches the Terminal with a UNREGISTRATION OK HLTP message.
SIP: *Component involved: 3, Exchanged messages: 2.*

6 Evaluation results

In this section we analyze the results of our evaluation with respect to the performance and modifiability/extensibility attributes performed on two software architectures that represent the same system, GSM on the Net. These architectures differ in the telecommunication protocol used internally: the first one uses ITU-T H.323, the second one IETF SIP.

The fixed scenario is the basic call, but in reality we factorized this scenario in different mini-scenarios, one for each phase of the call, to make the evaluation easier and more detailed. We used different metrics for each attribute: performance is measured in terms of number of components involved for each mini-scenario, and number of messages exchanged in the same mini-scenario. The modifia-

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

	COMPONENTS		MESSAGES	
	H.323	*SIP*	*H.323*	*SIP*
Discovery	4	0	2	0
Registration	4	3	2	2
Call Setup	8	8	8	8
Capabilities Exchange	8	0	4	0
Call Release	8	6	1	1
Unregistration	4	3	2	2

Table 1. Components involved and exchanged messages in a basic call

bility/extensibility is measured in terms of number of components affected by an addition of a supplementary service. We show the evaluation for the most common services, and informally we make an estimation of the difficulty to add each service in the system, only looking at the sequence diagrams which describe the implementation of the same service with H.323 and SIP protocols.

For the **performance**, Table 1 shows that, **in five mini-scenarios out of six, the number of components involved in the communication for the architecture SIP based is lower than in the other architecture, and for the last one the involved components are the same number**: for the mini-scenario *Discovery*, the SIP architecture has neither components nor message exchanged, given that it is a phase not specified in SIP protocol.

Moreover, for mini-scenarios Registration, Call Setup, Call Release and Unregistration the number of exchanged message are the same, in the Discovery we earn two messages, but the very important result is the Capabilities Exchange phase (mini-scenario), that is performed for every call, where we earn four messages. The results summarized in Table 1 allow us to believe that the SIP architecture performs better than the H.323 one.

The system modifiability and extensibility are increased under different points of view. In general, with the centralization of Access Agent in the MAC, the system is more open to different kinds of terminal. It is very easy to extend the system with new protocols: it is only needed to insert the specific gateway at the Gateway Level of the MAC. Moreover the HLTP level divides the external protocols from the internal protocol, and it makes the system more modifiable: it is possible to change the internal protocol without changing every gateway, given that the gateways translate from proprietary protocol into HLTP.

In order to **insert supplementary services**, we have first compared the software architectures behaviors with respect to four services: Virtual Private Network / Private Numbering Plan (VPN/PNP), Calling Line Identity Presentation/Restriction (CLIP/CLIR), Originating Call Barring (OCB), Personal Agent / Call Forwarding (PA/CF), showing that their insertion is immediate and the components which have to be changed are clearly specified. Then we have proposed a new implementation for three services by using the SIP protocol: Unattended Call Transfer (UCT), Call Hold (CH) and Attended Call Transfer (ACT), and, while in the old architecture it is impossible to understand which components have to be changed, we have shown that by using the SIP-based architecture it is clear which components the ERI designers have to modify in order to insert these services and to extend the system. The results are proposed in the Table 2.

7 Conclusions

This work shows a case study in architectural revision for a telecommunication system due to the change of the internal communication protocol from ITU-T H.323 [1] to IETF SIP [9]. The old and the new architectures are evaluated with respect to important quality attributes such as performance, modifiability and extensibility, and the result of this evaluation is described.

A direct result of this work is a demonstration that also in the consolidated domain of the design of telecommunication systems a software architecture evaluation may help to take important decisions very early in the development phase, such as the adoption of a protocol rather than another one. In this case study, we have revised the software architecture of the Ericsson GSM on the Net system replacing the internal telecommunication protocol, and with an analysis at the software architecture level, we have validated the intuition that a protocol change can solve the problems of a real system. Besides, the evaluation of software architecture with respect to modifiability and extensibility allows the designers to create telecommunication systems with shorter *time-to-market* for adding supplementary services or in general new functionalities. This allows the system to become competitive in the difficult market of telecommunication services.

We show in the performance evaluation that even an easy metric based on number of exchanged messages and components involved in the communication is significant and valuable in order to produce convincingly performance results on large component based telecommunication systems, and for instance protocols.

In this work we have proposed the use of the protocol architecture of SIP, a very young and promising protocol, to create a telecommunication system over IP. This architecture is demonstrated to be modifiable and extensible in order to add new supplementary services, it reduces the time-to-market requirement, uses different internal protocols to have better integration between services and better general performances. The proposed architecture can now be used

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.

	H.323		SIP	
Supplementary services	**Modified components**	**Insertion Difficulty level**	**Modified components**	**Insertion Difficulty level**
VPN/PNP	SA in the SN	Low	SA in the Proxt	Low
CLIP/CLIR	SA in the SN	Low	SA in the Proxy	Low
OCB	SA in the SN	Low	SA in the Proxy	Low
PA/CF	SA in the SN	Average	CM in the Proxy, Registrar	Low
UCT	?	High	UAC/UAS in the MAC CM in the Proxy	Low
CH	?	High	UAC/UAS in the MAC CM in the Proxy	Average
ACT	?	High	UAC/UAS in the MAC SA and CM in the Proxy	Average

Table 2. System extension respect to the supplementary services

as a pattern for the systems with the same performance and extensibility requirements.

Moreover, we have tried to adapt various approaches existing in the literature to the actual Ericsson life cycle development. This work shows that UML can be proficiently used as an architectural description language. Our current work focuses on the design and development of tool to support different kinds of quantitative analysis on telecommunication systems.

References

[1] Packed-based multimedia communication systems. *ITU-T H.323 Recommendation version 2, Official Specification*.

[2] F. Andolfi, F. Aquilani, S. Balsamo, and P. Inverardi. Deriving performance models of software architectures from message sequence charts. *IEEE Proceedings of Second Internationl Workshop on Software and Performance, Ottawa, Canada*, September 2000.

[3] F. Aquilani, S. Balsamo, and P. Inverardi. Performance analysis at the software architecture design level. *PERFORMANCE EVALUATION*, 45(2-3), July 2001.

[4] L. Bass, P. Clemens, and R. Kazman. *Software Architectures in Practice*. Addison-Wesley, 1998.

[5] P. Bengtsson, N. Lassing, J. Bosch, and H. van Vliet. Analyzing software architecture for modifiability. *HK/R Research Report*, 2000.

[6] J. Bosch and P. Molin. Software architecture design: Evaluation and transformation. *Proceedings of IEEE Engineering and Computer Systems Symposium*, December 1999.

[7] M. Castaldi, P. Inverardi, and S. Afsharian. A case study in performance, modifiability and extensibility analysis of a telecommunication system software architecture. *Technical Report, University of L'Aquila Archive*, July 2002.

[8] I. Dalgic and H. Fang. Comparison of h.323 and sip for ip telephony signaling. *Proceedings on Photonics East, Boston, MS*, September 1999.

[9] M. Handley, H. Shulzrinne, E. Schooler, and J. Rosemberg. Sip: Session initiation protocol. *Rfc 2543*, March 1999.

[10] C. Hofmeister, R. Nors, and D. Soni. *Applied Software Architectures*. Addison-Wesley, 1999.

[11] N. Kausar and J. Crowcroft. An architecture of conference control functions. *Proceedings on Photonics East, Boston, USA*, September 1999.

[12] R. Kazman, G. Abowd, L. Baa, and P. Clemens. Scenario-based analysis of software architectures. *Technical Report*, 1999.

[13] R. Kazman, M. Klein, and P. Clemens. Atam: Method for architecture evaluation. *Technical Report*, 2000.

[14] O. Oskarsson. Mechanisms of modifiability in large software systems. *Limkoping Studies in Science Technology Dissertation*, 77, 1982.

[15] D. Petriu, C. Shousha, and A. Jalnapurkar. Architecture based performance analysis applied to a telecommunication system. *IEEE Transactions on Software Engineering*, 26(11):1049–1065, November 2000.

[16] H. Shulzrinne. A comprehensive multimedia control architecture for the internet. *International Workshop on Network and Operating Systems Support for Digital Audioand Video, St. Louis*, May 1997.

[17] H. Shulzrinne, J. Lennox, and T. L. Porta. Implementing intelligent network services with session initiation protocol. *Columbia University Computer Science Technical Report CUCS-002-99*, January 1999.

[18] H. Shulzrinne and J. Rosemberg. The session initiation protocol: Providing advanced telephony services across the internet. *Bell Labs Technical Journal*, 3, November/December 1998.

[19] R. Sparks, C. Cunningham, A. Johnston, S. Donovan, and K. Summers. Sip telephony service examples with call flows. *Internet Draft*, October 1999.

[20] G. Team. Gsm on the net overview. *Ericsson Internal Report*, 2000.

[21] L. Williams and C. Smith. Performance evaluation of software architectures. *Proceedings of WOSP1998*, September 1998.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 04,2026 at 15:24:01 UTC from IEEE Xplore. Restrictions apply.